成　绩	
评阅人	

复 旦 大 学

本 科 生 课 程 论 文

论文题目：　基于深度学习的肥胖风险预测

修读课程：　人工智能的编程基础 AIE310006.02

选课学期：　2024-2025 学年第 2 学期

选课学生：　王泽扬 24301020009

完成日期：　2025.6.20

Contents

摘要 **3**

一、任务与数据分析 **3**

1.1 任务描述. **3**

1.2 数据分析. **3**

二、算法设计 **5**

2.1 数据预处理. **5**

2.2 神经网络模型设计. **6**

2.3 模型训练. **8**

三、实验结果与分析 **8**

3.1 实验设置. **8**

3.2 实验结果. **9**

四、结论 **10**

五、附录 **10**

基于深度学习的肥胖风险预测

【摘要】本项目基于 Kaggle 提供的数据集，构建深度学习模型实现肥胖风险的多类别自动预测与分类。数据集包含 20758 条训练样本和 13840 条测试样本，涵盖 16 个特征及 7 种肥胖等级。模型设计采用两层神经网络架构，损失函数为交叉熵，优化器选用 Adam（学习率 0.001）。实验结果显示，验证集最高准确率达 88.44%，Kaggle 测试集公私榜准确率分别为 85.69%和 85.49%，验证了模型有效性。

一、任务与数据分析

1.1 任务描述

本次任务的目标是根据每一个预测样本的 16 种特征预测其肥胖风险类别。肥胖风险类别包括 7 个等级，属于多分类问题。数据集包括了 train.csv 和 test.csv, train.csv 是训练数据集，包含 20,758 条样本，每条样本包含个体的特征和目标类别。test.csv 是测试数据集，包含 13,840 条样本，仅包含特征，需预测目标类别。

数据集除 id 外一共有 16 种特征，分别是 Gender, Age, Height, Weight, family_history_with_overweight, FAVC, FCVC, NCP, CAEC, SMOKE, CH2O, SCC, FAF, TUE, CALC,MTRANS ，而 NObeyesdad 则是需要预测的目标分类特征，它包含了 7 个分类等级，分别是 Insufficient_Weight, Normal_Weight, Overweight_Level_I, Overweight_Level_II, Obesity_Type_I, Obesity_Type_II, Obesity_Type_III。

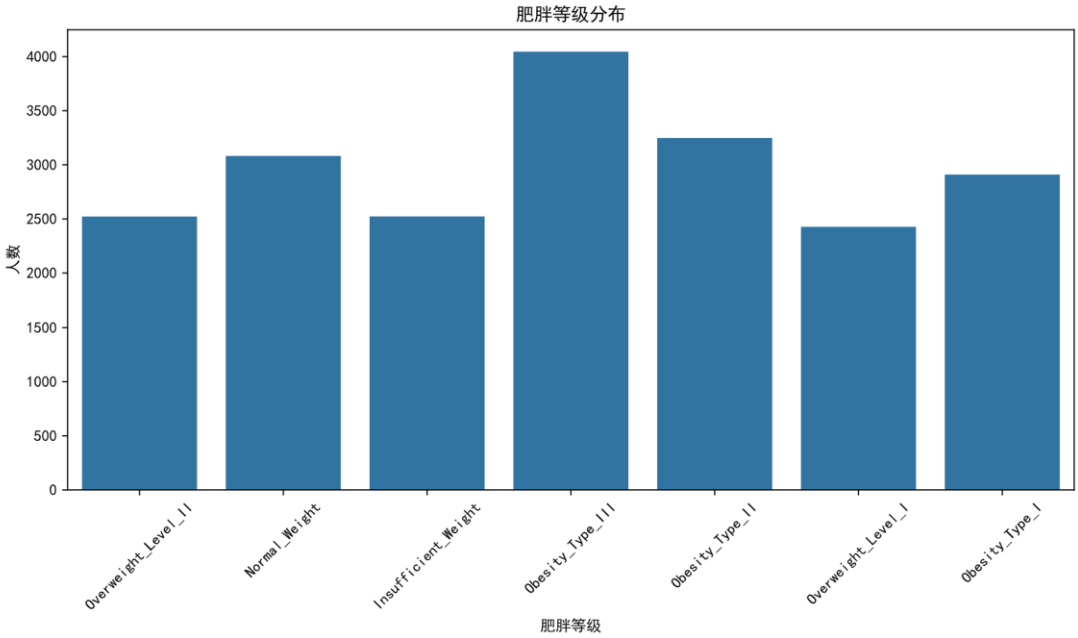
1.2 数据分析

对训练数据集进行分析后，主要发现如下：

1. 数据的特征分为两类，一类是基于数字的特征，如 age，height 等，它们有具体的数值；一类是离散的类别类特征，比如 FAVC 和 SCC 等，它们只有“yes”和“no”之分，或是“male”和“female”之分。因此，对于第一种特征，因为其数值特征的取值范围差异较大（如 age 都是两位数，

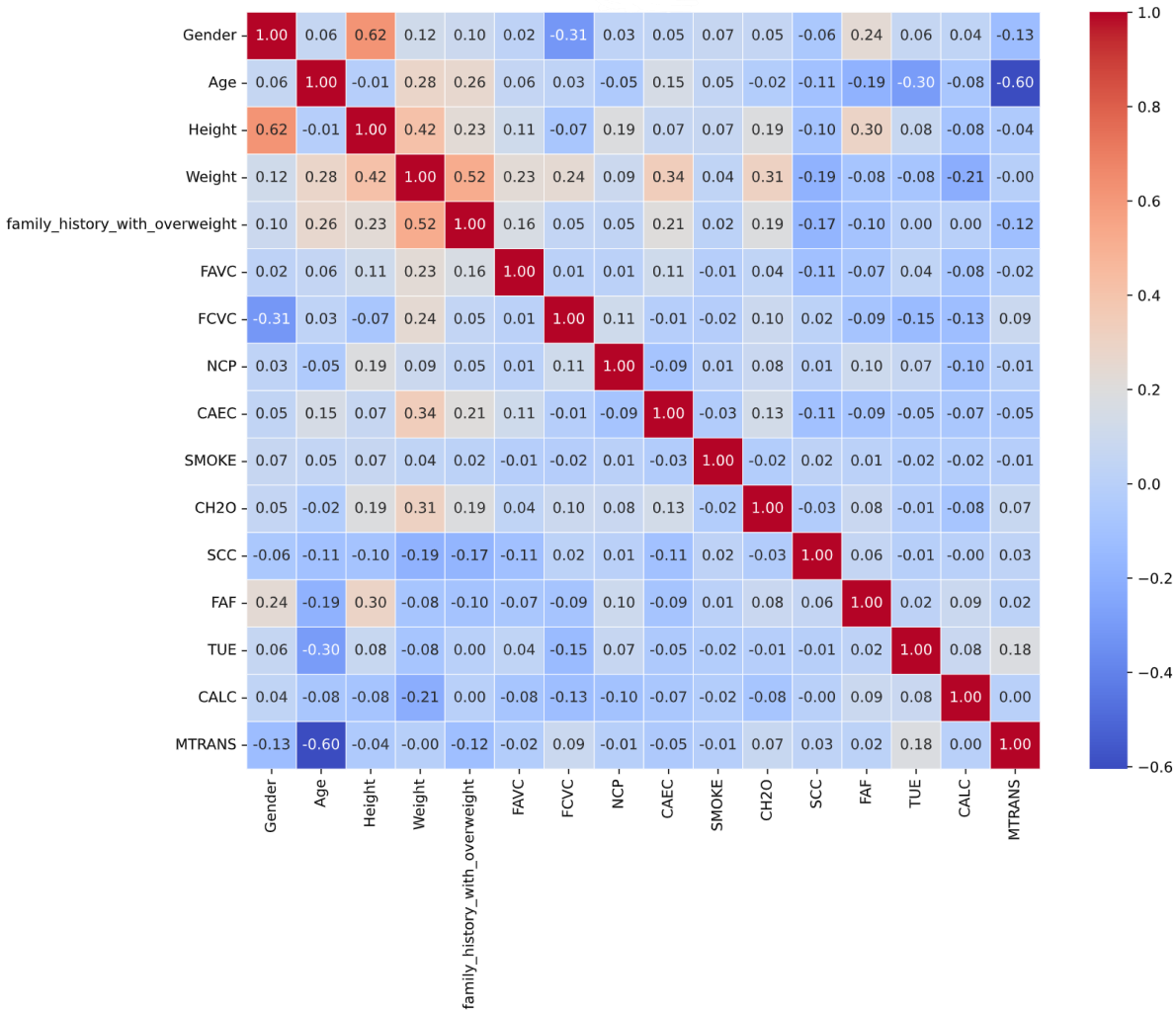
而 CH2O 在 0 到 3 之间），所以需要进行标准化处理。对于第二种特征，应该将字符串类型的分类变量转换为数值类型，使模型能够处理这些分类变量。

2. 通过对数据的可视化分析（如下图所示），我们发现，数据集中的 7 种肥胖类型的数据分布较均匀，每种类型的人数大致在 2500~4000 人左右。



（对肥胖等级类别分布的可视化分析）

3. 通过对训练数据集的 16 个特征进行热力图绘制（如下图所示）后，我们发现：除了少数特征之间相关性较高（比如 Height 与 Weight、Gender 与 MTRANS、Gender 与 Height 等），大多数特征之间的相关系数都在-0.2 到 0.2 之间，说明它们之间的线性相关性较弱，特征之间相对独立。由于大部分特征相关性不强，说明模型可以从多维度独立地学习特征对肥胖风险的影响，所以我们选择用神经网络模型进行数据学习。



（对 16 个特征的热力图数据分析）

二、算法设计

2.1 数据预处理

```python
# 数据预处理函数
def preprocess_data(df, is_train=True):
    # 处理分类特征
    cat_features = ['Gender', 'family_history_with_overweight', 'FAVC', 'CAEC', 'SMOKE', 'SCC', 'CALC', 'MTRANS']
    for col in cat_features:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])

    # 标准化数值特征
    num_features = ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE']
    scaler = StandardScaler()
    df[num_features] = scaler.fit_transform(df[num_features])

    if is_train:
        X = df.drop(['NObeyesdad', 'id'], axis=1)
        y = le.fit_transform(df['NObeyesdad'])
        return X, y, scaler, le
    else:
        return df.drop('id', axis=1), scaler
```

我们通过 preprocess_data 函数进行了数据预处理，preprocess_data 函数有两

个形参，df 和 is_train，df 是需要预处理的数据集，而 is_train 用于判断这个数据集需不需要训练，如果是 train.csv 训练数据的话，下面的程序输入的 is_train=True，需要训练，函数会删除目标列 NObeyesdad 和序号列 id，并返回特征矩阵 X、目标变量 y、标准化器 scaler 和标签编码器 le；如果是 test.csv 的话，is_train=False，不需要训练，函数会删除样本 ID 列（id）、返回标准化后的特征矩阵 X 和标准化器 scaler。

在 preprocess_data 函数中，我们先使用了 sklearn.preprocessing 模块里的 LabelEncoder 类对离散的类别类特征（比如 gender、FAVC 等）进行数值化处理。然后再使用 sklearn.preprocessing 模块里的 StandardScaler 类对数值特征进行标准化处理，以消除特征量纲不同带来的的影响，让模型更快地收敛。

```python
# 预处理
X_train, y_train, scaler, label_encoder = preprocess_data(train_df)
X_test, _ = preprocess_data(test_df, is_train=False)

# 划分验证集
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, stratify=y_train)
```

在主函数调用了 preprocess_data 函数后，程序将训练数据集划分为训练集（80%）和验证集（20%），以评估模型的泛化性能。

2.2 神经网络模型设计

```python
# 神经网络模型
class ObesityClassifier(nn.Module):
    def __init__(self, input_size, num_classes):
        super(ObesityClassifier, self).__init__()
        self.fc1 = nn.Linear(input_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, num_classes)
        self.dropout = nn.Dropout(0.3)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.relu(self.fc2(x))
        x = self.dropout(x)
        x = self.fc3(x)
        return x
```

我们设计了一个多层神经网络模型，具体结构如下：

1. **输入层：**输入层的输入特征为 input_size（实际就是特征的数目：16）,对应预处理后的特征数量，输入数据为标准化后的数值特征和编码后的分类特征。
2. **隐藏层：**

   第一层：第一隐藏层包含 128 个神经元，激活函数为 ReLU。为了防止模型过拟合，我们加入了 Dropout(0.3)，随机丢弃部分的神经元。

   第二层：第二隐藏层包含 64 个神经元，同样使用 ReLU 激活函数，加入 Dropout(0.3)进一步防止过拟合。
3. **输出层：**7 个神经元（实际就是对应目标的数目：7）用于输出类别概率。

我们在设计模型的时候，曾考虑过设计两层隐藏层好还是三层隐藏层好，三层隐藏层有助于处理更复杂的数据，但是过拟合的风险更高。于是我们尝试在原来两层隐藏层的基础上，把 ObesityClassifier 模型的 fc3 改为： self.fc3 = nn.Linear(64, 32)，然后加了一个 fc4： self.fc4 = nn.Linear(32, num_classes)，把 fc4 作为输出层，fc3 改为隐藏层，并且修改了 forward 函数的前向传播。这样模型就变成三层隐藏层了。

但是运行后的结果是，两层隐藏层的预测精确率高于三层隐藏层。三层隐藏层出现了明显的过拟合现象，导致准确率并不高。

所以我们最后选择了性能更好，训练速度更快，过拟合风险较低的两层隐藏层结构。

```
# 初始化模型
model = ObesityClassifier(input_size=X_train.shape[1], num_classes=len(label_encoder.classes_))
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

模型的损失函数为 CrossEntropyLoss 交叉熵损失，优化器选择了 Adam，学习率 lr 设为 0.001。

我们选择 Adam 优化器是因为 Adam 的自适应学习率能自动为不同特征分配合适的更新幅度，对要处理的数据集中的稀疏特征（比如说 gender、FAVC 等）更鲁棒。同时， Adam 的动量机制能加速模型的初期收敛，快速学习不同类别之

间的复杂边界。

2.3 模型训练

```python
# 训练函数
def train_model(model, train_loader, val_loader, criterion, optimizer, epochs=50):
    best_acc = 0
    for epoch in range(epochs):
        model.train()
        for inputs, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        # 验证
        model.eval()
        val_loss = 0
        correct = 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                outputs = model(inputs)
                val_loss += criterion(outputs, labels).item()
                _, preds = torch.max(outputs, 1)
                correct += (preds == labels).sum().item()

        val_acc = correct / len(val_loader.dataset)
        if val_acc > best_acc:
            best_acc = val_acc
            torch.save(model.state_dict(), 'best_model.pth')

        print(f'Epoch {epoch+1}/{epochs} - Val Loss: {val_loss:.4f} - Val Acc: {val_acc:.4f}')
    return best_acc
```

我们定义了 train_model 函数用来训练模型，并在每个 epoch 结束后评估验证集上的性能。为了防止过拟合，我们将验证集准确率最高的模型参数保存在 best_model.pth 文件。函数最后返回了 best_acc，也就是最佳验证集准确率，并在主程序上输出这个指标。

三、实验结果与分析

3.1 实验设置

1. **训练参数**：Epochs：50

   Batch Size：64

   learning rate：0.001

2. **实验环境：**

系统： Windows 11

Python 版本： Python 3.13.3 64-bit

Pytorch 版本： Pytorch 2.7.1+cpu

编辑器/IDE： VS Code 1.101.0 (user setup)

3.2 实验结果

1. **验证集性能：**

程序输出的最佳验证集准确率：0.884393063583815

验证集损失逐渐下降，模型收敛良好。

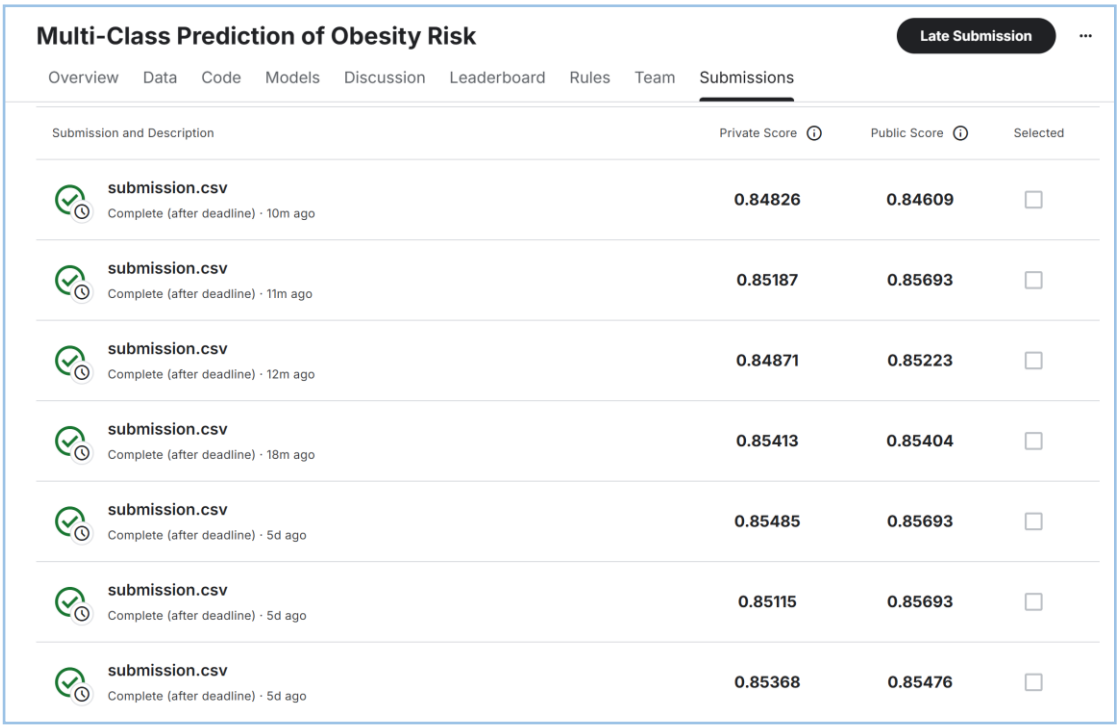
```
Epoch 46/50 - Val Loss: 21.9361 - Val Acc: 0.8789
Epoch 47/50 - Val Loss: 21.7355 - Val Acc: 0.8844
Epoch 48/50 - Val Loss: 21.8357 - Val Acc: 0.8805
Epoch 49/50 - Val Loss: 21.9945 - Val Acc: 0.8776
Epoch 50/50 - Val Loss: 21.9112 - Val Acc: 0.8839
best_acc: 0.884393063583815
```

（程序输出结果的最后几行）

2. **生成文件 Submission_csv 在 Kaggle 上的测试结果：**

Private Score 最高准确率：0.85485

Public Score 最高准确率：0.85693



（Kaggle 上的评测结果）

四、结论

本项目设计了一个基于深度学习的肥胖风险预测模型，并在模拟生成的数据集上进行了实验验证。模型在验证集上的最佳准确率达到了 88%，生成文件 Submission_csv 在 Kaggle 上测试的准确率达到了 85%，证明了深度学习在该任务中的有效性。通过实验分析，我们发现特征标准化、Dropout 层以及合理的模型架构设计对性能提升具有重要作用。

对于更加复杂，难度更大的数据分布，这个模型的学习能力肯定是不够的，我们认为针对未来更复杂的场景，可以尝试从以下方面改进我们的模型设计和训练流程：

1. 数据层面，面对所给的数据集中类别不平衡问题（比如某些类型数据更少），我们认为可以采用数据重采样技术提高模型对这些少数类的预测性能。
2. 算法与模型选择方面，可以尝试其他模型架构（如决策树类模型，它们通常对不平衡数据更鲁棒），并与深度学习模型进行对比。

五、附录：

完整代码见 Multi-Class Prediction of Obesity Risk.py